(1) What is a function, and why is it used in programming ? Explain the purpose of functions with an example of a simple C program that uses a function to calculate the square of a number.

→ A function is a block of code which only runs when it is called. You can pass data, known as parameters, into a function. Functions are used to perform certain actions, and they are important for reusing code. Define the code once, and use it many times.
We use functions for code reusability, Modularity, Maintainability, reduces complexity and testing, debugging.

- Code reusability: write a function once, use it multiple times
- Modularity: Programs are easier to understand when they are divided into functions
- Maintainability: Updating code in a function changes all instances where the function is called
- Reduces complexity: Instead of one long sequence of instructions, functions allow you to manage small tasks separately.
- Testing and Debugging: Easier to test and debug.

Program:

```
#include<stdio.h>

int square (int number)
{
 return number * number;
}
int main ()
{ int num;
  printf ("Enter a number:");
  scanf ("%d", &num);

int result = square(num);
printf ("The square of %d is %d \n", num, result);
return 0;
}
```

Output:

```
Enter a number: 5
The square of 5 is 25
```

2) Discuss how values are passed between functions. Write a program that demonstrates passing arguments to a function. and explain the scope of rule of functions with respect to local and global variables.

=> In programming, values are passed between functions primarily through parameters and return values. Functions in C follow a clear structure, which involves declaration, definition and calling.

Types of values passed between functions

i) With No return and No parameter

- This type of function performs a task but does not return any value or take any parameters.

Syntax:

```
void functionName()
{ // code execute
}
```

ii) With parameter and No return

- This type of function takes parameters but does not return any value

Syntax:

```
VoidfunctionName (type parameter 1, type parameter 2)
{
    // code to execute
}
```

iii) with return type and No parameter

- This type of function returns a value but does not take any parameters

Syntax:

```
return_type functionName()
{
    //code to execute
    return value;
}
```

iv) with return and parameters

- This type of function takes parameters and return a value

Syntax:

```
return_type functionName (type parameter1, type parameter2)
{
    //code to execute
    return value;
}
```

Passing Arguments

- In C, arguments are passed to functions in the order they are listed. The first argument corresponding to the first parameter, and so on.

Note: Always ensure that the number and type of arguments passed match the function's parameters.

program:

```
#include <stdio.h>
#include <conio.h>

void displayDetails (int age, float height)
{
printf("Age: %d, Height: %.2f\n", age, height);
}
int main ()
{
 clrscr();
displayDetails (25, 5.9);
getch();
return 0;
}
```

(Output)

Age: 25, Height: 5.90

Scope rule of functions

⇒ The scope of a variable refers to where in the program the variable can be accessed. variables can have local or global scope.

1) Local scope:

- Variables declared inside a function are local to that function. They cannot be accessed outside the function.

Example:

```
#include <stdio.h>
#include <conio.h>
void myfunction(){
 int localvar=5;
printf ("Local variable : %d\n", localvar);
}
int main(){
clrscr();
myfunction();
getch();
return 0; }
```

output:

Local variable : 5

7) Global Scope

- variables declared outside all functions are global variables and co
and accessed from any function in the program.

Example:

```
#include <stdio.h>
#include <conio.h>
int globalvar = 10;

void myfunction()
{ printf("Global variable: %d\n", globalvar);
}
int main()
clrscr()
myfunction();
getch();
return 0;
}
```

output

Global variable : 10

③ Explain the order of passing arguments in functions. Create a program that passes multiple arguments to a function and discuss how the order in which the arguments are passed affect the output.

→ In C, arguments are passed to functions in the order they are listed. The first argument corresponding to the first parameter, and so on.
Always ensure that the number and type of arguments passed match the function's parameters.

Let's take an example program:

```
#include<stdio.h>

Void calculateResults(int marks1, int marks2, int marks3, int totalmarks)
{
  int total= marks1 + marks2 + marks3;
  float average = total /3.0;
  float percentage = (total / (float)totalmarks) * 100;

  printf("Total : %.d \n", total);
  printf("Average: %.2f\n", average);
  printf("Percentage: %.2f \n", percentage);
}
int main()
{
int Subject1 = 85, Subject2 = 90, Subject3 = 88;
int maxmarks = 300;
```

```
calculateResult(subject1, subject2, subject3, maxmarks);
return 0;
}
```

Output:

Total : 263
Average : 87.67
percentage : 87.67%

Discussion how order in which parameters passed affect output.

i) Argument matching by position:
In C, arguments are assigned to function parameter based on their order. The first argument is assigned to the first parameter, the second to the second and so on.

ii) Incorrect order leads to logical errors:
- Passing arguments in the wrong order may cause incorrect values to the used in calculations, leading to unexpected or incorrect results.

iii) Functions expect consistent Argument order:
- If the arguments are passed in an unexpected order, the function may produce erroneous outputs, especially if the parameters represents different concepts.

Q) What are library functions in C? provide an example of using standard library functions (like printf(), scanf(), strlen()) and explain their importance.

→ C provides several library functions (predefined functions) that can be used without needing to write them yourself. These are provided by header files like <stdio.h>, <math.h> etc.

* common library functions:
- printf() : To print output to the screen
- scanf() : To get input from the user
- strlen() : To find the length of a string.
- sqrt() : To calculate the square root.

Example:

```
#include<stdio.h>
#include <conio.h>
#include <string.h>
int main()
{ char str[50];
  int length;
  clrscr();
  printf("Enter a string");
  scanf("%s", str);
  length = strlen(str);
printf("Length of the string : %d\n", length);
getch();
return 0;
}
```

Importance of library functions:

i) Code reusability
- Library functions provide reusable, pre-written code for common tasks. This reduces the need to rewrite functionality, saving time and effort.

ii) Efficiency:
- Library functions are optimized for performance, often being more efficient than custom-written code. They ensure faster execution of common operations.

iii) Simplifies Development:
- Library functions abstract complex tasks like memory management or file handling. This allows developers to focus on the core logic of their application.

iv) Reduces Errors:
Library functions are well-tested and widely used, reducing the likehood of bugs. They ensure reliable and stable code with minimal debugging.

⑤ Explain call by value and call by Reference with examples. Write a program that demonstrate the difference between call by value and call by reference using pointers.

→ call by value: In call by values, a copy of the actual parameter's value is passed to the function. Changes made to the parameter inside the function do not affect the actual parameters.

Syntax: void functionName (dataType parameter)
{ // function body }

Example:

```
#include<stdio.h>
#include <conio.h>
void modifyValue(int num){
  num = num + 10;
}
int main(){
  clrscr();
  int x=5;
  printf("Before function call: %d\n", x);
  modifyValue(x);
  printf("After function call: %d\n", x);
  getch();
  return 0;
}
```

Output:

Before function call: 5
After function call: 5

2) Call by reference:
In call by reference instead of passing a copy of the actual parameter, a reference to the actual parameter is passed to the function. Changes made to the parameter inside the function affect the actual parameter.

Syntax:
```
Void functionName (data type * parameter){
    // function body }
```

Code:
```
#include<stdio.h>
#include <conio.h>

Void modifyValue(int * num){
 * num = * num + 10;
}
 int main()
{
 clrscr();
int x=5;
printf(" Before function call : %d\n", x);
modifyValue (&x);
printf(" After function call : %d\n", x);
getch();
return 0;
}
```

Output
Before function call : 5
After function call : 15

⇒ demonstrating the difference between call by value and call by reference using pointers.

```
#include<stdio.h>

void callbyvalue (int a)
{
  a=10;
}

void callbyreference (int *b)
{
   *b=10;
}

int main()
{ int x=5;
  int y=5;

printf(" Before callbyvalue, x=%d\n", x);
callbyvalue (x);
printf(" After callbyvalue, x=%d\n", x);

printf(" Before callbyreferene, y=%d\n", y);
callbyreference (&y);
printf(" After callbyreference, y=%d\n", y);

return 0;
}
```

Output

```
Before callbyvalue, x=5
After callbyvalue, x=5
Before callbyreference, y=5
After callbyreference, y=10
```

Explanation

i) call by value
- A copy of the variable x is passed to the callbyvalue function
- changes inside the function do not affect the original variable.

ii) call by reference
- The address of the variable y is passed to the callbyreference function using a pointer.
- Changes inside the function directly modify the original variable.